



TRƯỜNG ĐẠI HỌC
VĂN LANG

Đạo đức - Ý chí - Sáng tạo

KHOA CÔNG NGHỆ THÔNG TIN

CƠ SỞ LẬP TRÌNH

(Programming Basics)

CHAPTER 3: VARIABLES AND TYPES



GVGD: MSc. Van Trung NGUYEN



HỌC KỲ I



KHÓA:



1. Variables - Name, Type and Value

Computer programs manipulate (or process) data. A *variable* is used to *store a piece of data* for processing. It is called *variable* because you can change the value stored.

More precisely, a *variable* is a *named* storage location, that stores a *value* of a particular data *type*. In other words, a *variable* has a *name*, a *type* and stores a *value*.

- A variable has a *name* (aka *identifier*), e.g., radius, area, age, height and numStudents. The name is needed to *uniquely* identify and reference each variable. You can use the *name* to assign a value to the variable (e.g., radius = 1.2), and to retrieve the value stored (e.g., radius*radius*3.1419265).
- A variable has a *data type*. The frequently-used Java *data types* are:
 - int: meant for integers (whole numbers) such as 123 and -456.
 - double: meant for floating-point number (real numbers) having an optional decimal point and fractional part, such as 3.1416, -55.66, 1.2e3, or -4.5E-6, where e or E denotes exponent of base 10.
 - String: meant for texts such as "Hello" and "Good Morning!". Strings are enclosed within a pair of double quotes.
 - char: meant for a single character, such as 'a', '8'. A char is enclosed by a pair of single quotes.



- In Java, you need to declare the *name* and the *type* of a variable before using a variable. For examples,

```
int sum;           // Declare an "int" variable named "sum"  
double average;    // Declare a "double" variable named "average"  
String message;    // Declare a "String" variable named "message"  
char grade;        // Declare a "char" variable named "grade"
```

- A variable can store a *value* of the declared data *type*. It is important to take note that a variable in most programming languages is associated with a type, and can only store value of that particular type. For example, an `int` variable can store an integer value such as 123, but NOT floating-point number such as 12.34, nor string such as "Hello".
- The concept of *type* was introduced in the early programming languages to *simplify* interpretation of data made up of binary sequences (0's and 1's). The type determines the size and layout of the data, the range of its values, and the set of operations that can be applied.

The following diagram illustrates three types of variables: `int`, `double` and `String`. An `int` variable stores an integer (or whole number or fixed-point number); a `double` variable stores a floating-point number (or real number); a `String` variable stores texts.



TYPE	NAME	VALUE	
int	number	→ 1	Stored only Integer
int	sum	→ 500500	Stored only Integer
double	radius	→ 5.5	Stored only floating-point number
double	area	→ 95.0334	Stored only floating-point number
String	greeting	→ Hello	Stored only texts
String	statusMsg	→ Game Over	Stored only texts

*A variable has a **name**, stores a **value** of the declared **type**.*



An *identifier* is needed to *name* a variable (or any other entity such as a method or a class). Java imposes the following *rules on identifiers*:

- An identifier is a sequence of characters, of any length, comprising uppercase and lowercase letters (a-z, A-Z), digits (0-9), underscore (_), and dollar sign (\$).
- White space (blank, tab, newline) and other special characters (such as +, -, *, /, @, &, commas, etc.) are not allowed. Take note that blank and dash (-) are not allowed, i.e., "max value" and "max-value" are not valid names. (This is because blank creates two tokens and dash crashes with minus sign!)
- An identifier must begin with a letter (a-z, A-Z) or underscore (_). It cannot begin with a digit (0-9) (because that could confuse with a number). Identifiers begin with dollar sign (\$) are reserved for system-generated entities.
- An identifier cannot be a reserved keyword or a reserved literal (e.g., class, int, double, if, else, for, true, false, null).
- Identifiers are case-sensitive. A rose is NOT a Rose, and is NOT a ROSE.

Examples: abc, _xyz, \$123, _1_2_3 are valid identifiers. But 1abc, min-value, surface area, ab@c are NOT valid identifiers.

Caution: Programmers don't use *blank* character in any names (filename, project name, variable name, etc.). It is either not supported (e.g., in Java and C/C++), or will pose you many more challenges.



Variable Naming Convention

A variable name is a noun, or a noun phrase made up of several words with no spaces between words. The first word is in lowercase, while the remaining words are initial-capitalized. For examples, `radius`, `area`, `fontSize`, `numStudents`, `xMax`, `yMin`, `xTopLeft`, `isValidInput`, and `thisIsAVeryLongVariableName`. This convention is also known as *camel-case*.

Recommendations

1. It is important to choose a name that is *self-descriptive* and closely reflects the meaning of the variable, e.g., `numberOfStudents` or `numStudents`, but not `n` or `x`, to store the number of students. It is alright to use abbreviations.
2. Do not use *meaningless* names like `a`, `b`, `c`, `i`, `j`, `k`, `n`, `i1`, `i2`, `i3`, `j99`, `exercise85` (what is the purpose of this exercise?), and `example12` (What is this example about?).
3. Avoid *single-letter* names like `i`, `j`, `k`, `a`, `b`, `c`, which are easier to type but often meaningless. Exceptions are common names like `x`, `y`, `z` for coordinates, `i` for index. Long names are harder to type, but self-document your program. (I suggest you spend sometimes practicing your typing.)
4. Use *singular* and *plural* nouns prudently to differentiate between singular and plural variables. For example, you may use the variable `row` to refer to a single row number and the variable `rows` to refer to many rows (such as an array of rows - to be discussed later).



Variable Naming Convention

A variable name is a noun, or a noun phrase made up of several words with no spaces between words. The first word is in lowercase, while the remaining words are initial-capitalized. For examples, `radius`, `area`, `fontSize`, `numStudents`, `xMax`, `yMin`, `xTopLeft`, `isValidInput`, and `thisIsAVeryLongVariableName`. This convention is also known as *camel-case*.

Recommendations

1. It is important to choose a name that is *self-descriptive* and closely reflects the meaning of the variable, e.g., `numberOfStudents` or `numStudents`, but not `n` or `x`, to store the number of students. It is alright to use abbreviations.
2. Do not use *meaningless* names like `a`, `b`, `c`, `i`, `j`, `k`, `n`, `i1`, `i2`, `i3`, `j99`, `exercise85` (what is the purpose of this exercise?), and `example12` (What is this example about?).
3. Avoid *single-letter* names like `i`, `j`, `k`, `a`, `b`, `c`, which are easier to type but often meaningless. Exceptions are common names like `x`, `y`, `z` for coordinates, `i` for index. Long names are harder to type, but self-document your program. (I suggest you spend sometimes practicing your typing.)
4. Use *singular* and *plural* nouns prudently to differentiate between singular and plural variables. For example, you may use the variable `row` to refer to a single row number and the variable `rows` to refer to many rows (such as an array of rows - to be discussed later).

2.3 Variable Declaration

To use a variable in your program, you need to first *introduce* it by *declaring* its *name* and *type*, in one of the following syntaxes. The act of declaring a variable allocates a storage of size capable of holding a value of the type.

Syntax	Example
<pre>// Declare a variable of a specified type type identifier;</pre>	<pre>int sum; double average; String statusMsg;</pre>
<pre>// Declare multiple variables of the SAME type, // separated by commas type identifier1, identifier2, ..., identifierN;</pre>	<pre>int number, count; double sum, difference, product, quotient; String helloMsg, gameOverMsg;</pre>
<pre>// Declare a variable and assign an initial value type identifier = initialValue;</pre>	<pre>int magicNumber = 99; double pi = 3.14169265; String helloMsg = "hello,";</pre>
<pre>// Declare multiple variables of the SAME type, // with initial values type identifier1 = initValue1, ..., identifierN = initValueN;</pre>	<pre>int sum = 0, product = 1; double height = 1.2, length = 3.45; String greetingMsg = "hi!", quitMsg = "bye!";</pre>



Take note that:

- A variable is declared with a *type*. Once the *type* of a variable is declared, it can only store a value belonging to that particular type. For example, an `int` variable can hold only integer (such as 123), and NOT floating-point number (such as -2.17) or text string (such as "Hello").
- Each variable can only be declared once because identifier shall be unique.
- You can declare a variable anywhere inside the program, as long as it is declared before being used.
- The type of a variable cannot be changed inside the program, once it is declared.
- A variable declaration statement begins with a *type*, and works for only that type. In other words, you cannot declare variables of two different types in a single declaration statement.
- Java is a *statically-typed* language. This means that the type is resolved at compile time and never changes.



2.4 Constants (final variables)

Constants are *non-modifiable (immutable)* variables, declared with keyword `final`. You can only assign values to final variables ONCE. Their values cannot be changed during program execution. For examples:

```
final double PI = 3.14159265; // Declare and initialize the constant

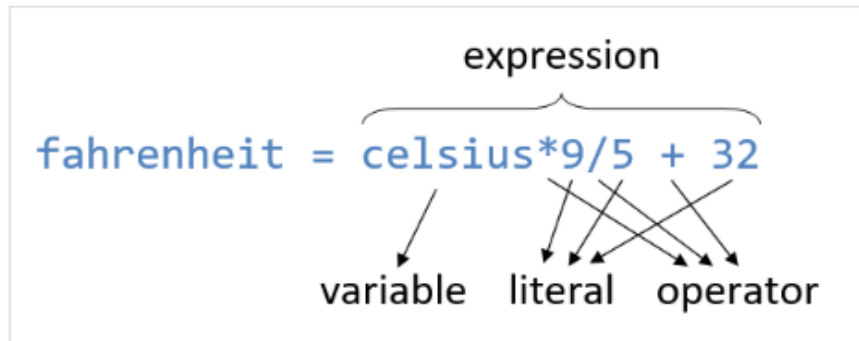
final int SCREEN_X_MAX = 1280;
SCREEN_X_MAX = 800; //compilation error: cannot assign a value to final variable

// You can only assign value to final variables ONCE
final int SCREEN_Y_MIN;
SCREEN_Y_MIN = 0; // First assignment
SCREEN_Y_MIN = 10; //compilation error: variable might already have been assigned
```

Constant Naming Convention: Use uppercase words, joined with underscore. For example, `MIN_VALUE`, `MAX_SIZE`, and `INTEREST_RATE_6_MTH`.

2.5 Expressions

An *expression* is a combination of *operators* (such as '+' and '-') and *operands* (variables or literals), that can be *evaluated to yield a single value of a certain type*.



For example,

```
// "int" literals
((1 + 2) * 3 / 4) % 6    // This expression is evaluated to an "int" value

// "double" literals
3.45 + 6.7               // This expression is evaluated to a "double" value

// Assume that variables sum and number are "int"
sum + number * number    // evaluates to an "int" value

// Assume that variables principal and interestRate are "double"
principal * (1.0 + interestRate) // evaluates to a "double" value
```

2.6 Assignment (=)

An *assignment statement* evaluates the RHS (Right-Hand Side) and assigns the resultant value to the variable of the LHS (Left-Hand Side).

The syntax for assignment statement is:

Syntax	Example
<pre>// Assign the RHS literal value to the LHS variable variable = literalValue;</pre>	<pre>int number; number = 9;</pre>
<pre>// Evaluate the RHS expression and assign the result to the LHS variable variable = expression;</pre>	<pre>int sum = 0, number = 8; sum = sum + number;</pre>

The assignment statement should be interpreted this way: The *expression* on the RHS is first evaluated to produce a resultant value (called *r-value* or right-value). The *r-value* is then assigned to the variable on the left-hand-side (LHS) or *l-value*. Take note that you have to first evaluate the RHS, before assigning the resultant value to the LHS. For examples,

```
int number;  
number = 8;           // Assign RHS literal value of 8 to the LHS variable number  
number = number + 1;  // Evaluate the RHS expression (number + 1),  
                      // and assign the resultant value back to the LHS variable number  
  
8 = number;           // Invalid in Programming, LHS shall be a variable  
number + 1 = sum;     // Invalid in Programming, LHS shall be a variable
```



'=' is Assignment, NOT Equality

In Java, the equal symbol '=' is known as the *assignment operator*. The meaning of '=' in programming is different from Mathematics. It denotes *assignment of the RHS value to the LHS variable*, NOT *equality of the RHS and LHS*. The RHS shall be a literal value or an expression that evaluates to a value; while the LHS must be a variable.

Note that $x = x + 1$ is valid (and often used) in programming. It evaluates the RHS expression $x + 1$ and assigns the resultant value to the LHS variable x . On the other hand, $x = x + 1$ is illegal in Mathematics.

While $x + y = 1$ is allowed in Mathematics, it is invalid in programming because the LHS of an assignment statement shall be a variable.

Some programming languages use symbol ":", ">" or "<" as the assignment operator to avoid confusion with equality.

```
int x = 79;  
x = x + 1;
```

START



Exercise

Example (Variable Names and Types): Paul has bought a new notebook of "idol" brand, with a processor speed of 2.66GHz, 8 GB of RAM, 500GB hard disk, with a 15-inch monitor, for \$1760.55. He has chosen service plan 'C' among plans 'A', 'B', 'C', and 'D', plus on-site servicing but did not choose extended warranty. Identify the data types and name the variables.

The possible variable names and types are:

```
String name = "Paul";
String brand = "idol";
double processorSpeedInGHz = 2.66; // or float
double ramSizeInGB = 8;           // or float
int harddiskSizeInGB = 500;       // or short
int monitorInInch = 15;          // or byte
double price = 1760.55;
char servicePlan = 'C';
boolean onSiteService = true;
boolean extendedWarranty = false;
```

Exercise (Variable Names and Types): You are asked to develop a software for a college. The system shall maintain information about students. This includes name, address, phone number, gender, date of birth, height, weight, degree pursued (e.g., B.Sc., B.A.), year of study, average GPA, with/without tuition grant, is/is not a scholar. Each student is assigned a unique 8-digit number as id.

You are required to identify the variables, assign a suitable name to each variable and choose an appropriate type. Write the variable declaration statements as in the above example.



TRƯỜNG ĐẠI HỌC
VĂN LANG

Đạo đức - Ý chí - Sáng tạo

KHOA CÔNG NGHỆ THÔNG TIN

